

Programmer avec les suggestions orthographiques

Contrats, tables, images et composition de fonctions

Traduit et adapté depuis Bootstrap World (Fall 2026)

Suite de la leçon « Algorithmes pilotés par les données »

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Introduction aux contrats	3
1.1	Lancement	3
1.2	Trier et raccourcir les tables	4
1.2.1	Trier les résultats	4
1.2.2	Raccourcir les résultats	5
1.3	Visualiser les tables	5
1.3.1	Tables de fréquence, diagrammes en barres et camemberts	5
1.3.2	Les vrais correcteurs renvoient des mots, pas des tables!	6
1.4	Synthèse	7
2	Composer avec les Cercles d'Évaluation	8
2.1	Lancement	8
2.2	Cercles d'Évaluation	8
2.2.1	Observer et s'interroger	9
2.2.2	À vous de jouer	9
2.3	Interface du correcteur orthographique	9
2.4	Synthèse	11

Introduction aux contrats

Objectif de la section

En s'appuyant sur la leçon précédente (*Algorithmes pilotés par les données*), les élèves découvrent les fonctions Pyret pour travailler avec des tables : *trier*, *raccourcir*, *visualiser* et *extraire* des données. Ils sont introduits aux **contrats** comme « mode d'emploi des fonctions » (nom, domaine, image).

Lancement

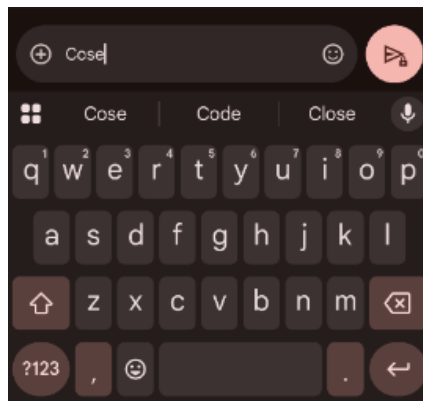


FIGURE 1 – Capture d'une application de messagerie proposant des alternatives pour un mot possiblement mal orthographié.

Activité

Assurez-vous d'avoir ouvert le fichier **Correcteur-Orthographique-Starter.arr** dans l'éditeur Pyret, et que votre *Zone de Définitions* contient :

```
resultats = alt-words("plation", WORDS-L-FR, 4)
```

Cliquez sur « Run », puis évaluez `resultats` dans la Zone d'Interactions : vous devez voir une table avec deux colonnes, `word` et `edit-distance`, contenant les suggestions de correction orthographique de "plation" à distance d'édition ≤ 4 selon le dictionnaire français WORDS-L-FR (40 000 mots).

Notre correcteur nous donne beaucoup de données à explorer, sous forme d'une **table** de mots suggérés et de leurs distances d'édition. Nous savons déjà sauvegarder cette table dans une variable (`resultats`) pour éviter de la recalculer à chaque fois.

Mais peut-on faire quelque chose d'*intéressant* avec ces données ?

Réflexion

Un correcteur ne nous montre pas une table de 500 suggestions. Notre téléphone ne nous affiche pas la distance d'édition de chaque mot proposé. Mais ce n'est pas parce que la distance d'édition n'est pas affichée qu'elle n'est pas utilisée : le téléphone s'en sert pour choisir les mots *les plus probables* à suggérer.

- Si vous avez un téléphone sous la main, essayez de taper « plation » dans une nouvelle conversation. Sinon, ouvrez un document texte et tapez « plation ».
- Quelles orthographes alternatives votre appareil propose-t-il ?

Nous voulons prendre le résultat de `alt-words` et *lui faire quelque chose*, exactement comme le font les téléphones et les correcteurs.

Trier et raccourcir les tables

Trier les résultats

Les erreurs à 1 typo sont plus fréquentes que les erreurs à 2 ou 3 typos. Nous aimerions donc **trier** les suggestions, en mettant les plus probables en haut.

Activité

1. Testez ces deux expressions dans la Zone d'Interactions et notez ce que vous observez :

```
sort(resultats, "edit-distance", false)
sort(resultats, "word", true)
```

2. Vous avez déjà vu les types `Number`, `String` et `Boolean`. Quel type de données la fonction `sort` produit-elle ?
3. La fonction `sort` prend trois arguments. Décrivez-les dans un tableau (type + information contenue).

Il existe beaucoup d'autres fonctions dans Pyret. On les décrit à l'aide de **contrats**. Le contrat de `sort` est :

```
# sort :: (Table t, String colonne, Boolean croissant)
-> Table
```

Chaque contrat contient trois éléments :

1. **Nom** — le nom de la fonction, que l'on écrit chaque fois qu'on veut l'utiliser.
2. **Domaine** — le(s) type(s) de données que la fonction *consomme* (ses arguments).
3. **Image (Range)** — le type de donnée que la fonction *produit*.

Les contrats sont le *mode d'emploi* des fonctions.

Activité

4. Quel code permet de trier `resultats` par ordre alphabétique, de A à Z ?

Raccourcir les résultats

Quand on fait une faute de frappe, notre téléphone ne nous affiche que 3 à 5 suggestions. Nous aimerions **raccourcir** la table pour ne garder que les premières lignes.

Activité

4. Tapez `first-n-rows(resultats, 10)`. Que se passe-t-il?
5. Si nous voulions une table des 5 premières lignes de `resultats`, quel code écrivions-nous?
6. Combien d'arguments composent le *Domaine* de `first-n-rows`? Quel est son *Image*?
7. En vous inspirant du contrat de `sort`, écrivez le contrat de `first-n-rows` :

```
nom_fonction :: Domaine -> Image
```

8. Pyret possède aussi une fonction `last-n-rows`. **Faites une prédiction** : quel est son contrat?
9. Testez-la dans Pyret! Votre prédiction était-elle juste?

Visualiser les tables

Tables de fréquence, diagrammes en barres et camemberts

Activité

1. Faites défiler tous les mots suggérés dans `resultats`. Quelle `edit-distance` est la plus fréquente? La moins fréquente?
2. Évaluez `count(resultats, "edit-distance")`. Que renvoie Pyret?
3. Écrivez le contrat de la fonction `count` :

```
nom_fonction :: Domaine -> Image
```

Le contrat de la fonction `bar-chart` est :

```
# bar-chart :: Table, String -> Image
```

Activité

4. Réalisez un diagramme en barres de votre table `resultats`, en utilisant `"edit-distance"` comme colonne. Qu'obtenez-vous?
5. Placez votre curseur sur l'une des barres. Quelle information cela vous donne-t-il?
6. Comment cette information se relie-t-elle à la table renvoyée par `count`?
7. Essayez de faire un diagramme en barres utilisant la colonne `"word"`. Cette visualisation est-elle *utile*? Pourquoi?

Le contrat de la fonction `pie-chart` est :

```
# pie-chart :: Table, String -> Image
```

Activité

4. Réalisez un *camembert* de votre table `resultats`, en utilisant `"edit-distance"` comme colonne.

Les vrais correcteurs renvoient des mots, pas des tables!

Définissons une ligne. En bas de la Zone de Définitions, tapez :

```
ligne2 = row-n(resultats, 1)
```

Cliquez sur « Run ».

Activité

1. Évaluez `ligne2` dans la Zone d'Interactions. Qu'obtenez-vous?
2. Avec vos propres mots, expliquez ce que fait `row-n`.
3. Complétez le contrat de `row-n` (l'Image est donné : Row).

```
row-n :: Domaine -> Row
```

4. Que renvoient `ligne2["word"]` et `ligne2["edit-distance"]`?
5. Évaluez `row-n(resultats, 1)["word"]`. Qu'obtenez-vous?

Un code comme `ligne2["word"]` ou `ligne2["edit-distance"]` s'appelle un **accesseur de ligne** (*row accessor*). Il permet d'extraire la valeur d'une cellule précise de la table.

Activité

4. Avec vos propres mots, que font les accesseurs de ligne?
5. Quel code produit le *premier* mot suggéré par la table?
6. Comment obtenir la *edit-distance* du *cinquième* mot suggéré par la table?

Notes pour l'enseignant·e

À ce stade, les élèves devraient être à l'aise avec : `row-n` → Row, `row["colonne"]` → valeur de cette cellule. Bien insister sur la numérotation qui commence à 0 : `row-n(t, 0)` renvoie la *première* ligne.

Synthèse

Réflexion

- `first-n-rows` et `last-n-rows` ont *exactement* le même **Domaine** et la même **Image**, mais font des choses différentes. Il en va de même pour `circle` et `triangle`. Comment est-il possible d'avoir le même **Domaine** *et* la même **Image**, tout en se comportant si différemment ?
 - Ce qu'une fonction a **besoin** n'est pas ce qu'elle **fait**.
- Pourquoi `sort` a-t-il besoin de plus d'arguments dans son **Domaine** que les deux autres ?
 - Parce qu'elle a besoin de *plus d'informations* : la colonne et l'ordre.
- Pourquoi une application de correction orthographique aurait-elle besoin d'utiliser des accesseurs de ligne ?
 - Parce qu'on veut suggérer *un mot*, pas toute une ligne de la table !
 - Et on peut vouloir tenir compte de la `edit-distance` pour faire cette suggestion.

Point clé

Contrat : le mode d'emploi d'une fonction.

$\text{nom} :: \text{Domaine} \rightarrow \text{Image}$

Le **Domaine** décrit ce que la fonction consomme. L'**Image** décrit ce qu'elle produit. Deux fonctions peuvent avoir le même contrat et faire des choses différentes : le contrat dit *quoi*, pas *comment*.

Composer avec les Cercles d'Évaluation

Objectif de la section

Les élèves apprennent à combiner plusieurs fonctions en un seul programme, en utilisant les **Cercles d'Évaluation** : une représentation visuelle de la structure de l'expression, indépendante de la syntaxe.

Lancement

Votre téléphone ne vous montre pas toutes les suggestions possibles :

- il n'en affiche que les 3 à 5 premières ;
- il les trie en général par *edit-distance* croissante (les erreurs les plus petites d'abord).

Réflexion

Comment pourrions-nous combiner les fonctions que nous avons déjà vues pour reproduire ce comportement ? Discutez-en avec vos voisin-es.

Si nous voulons *trier et raccourcir* notre table :

- En regardant l'**Image** de `sort`, on sait qu'elle produit une `Table` :

```
# sort :: (Table t, String colonne, Boolean  
croissant) -> Table
```

- En regardant le **Domaine** de `first-n-rows`, on sait qu'elle *consomme* une `Table` :

```
# first-n-rows :: (Table t, Number n) -> Table
```

Mais comment faire travailler `sort` et `first-n-rows` *ensemble* ?

Cercles d'Évaluation

Une façon d'organiser nos pensées est de **diagrammer** ce que nous voulons faire. Les **Cercles d'Évaluation** nous aident à réfléchir à *quoi* faire, sans nous soucier de la syntaxe (guillemets, parenthèses, etc.). Ils nous permettent d'utiliser tout notre cerveau pour *réfléchir*, avant de l'utiliser pour *coder*.

Les règles sont simples :

1. Chaque Cercle contient **une** — **et une seule** — fonction, écrite en haut.
2. Les arguments de la fonction sont écrits de gauche à droite, au milieu du Cercle.
3. Les valeurs (Nombres, Strings, Booléens) sont écrites telles quelles. Ce n'est que lorsqu'on veut *utiliser une fonction* que l'on dessine un Cercle imbriqué.
4. Les Cercles peuvent contenir d'autres Cercles !

Observer et s'interroger

Supposons que nous voulons voir les 5 premiers résultats de `resultats`, triés par `edit-distance` croissante. Commençons par générer la table triée, puis utilisons `first-n-rows` pour prendre les 5 premières.



En code Pyret :

```
sort(resultats, "edit-distance", true)
first-n-rows(sort(resultats, "edit-distance", true), 5)
```

Activité

1. Que **remarquez**-vous ?
2. Que **vous demandez**-vous ?

Pour convertir un Cercle d'Évaluation en code, on part de l'*extérieur* et on va vers l'*intérieur*. Après le nom de chaque fonction, on ouvre une paire de parenthèses, puis on convertit chaque argument contenu dans le Cercle.

À vous de jouer

Supposons que vous vouliez que l'ordinateur prenne les 200 premières lignes de `resultats`, puis les trie par `edit-distance décroissante`.

Activité

1. **Prendre les 200 premières lignes** : dessinez le Cercle d'Évaluation correspondant.
2. **Trier par edit-distance décroissante** : dessinez le Cercle d'Évaluation qui prend le précédent en argument.
3. Convertissez le premier Cercle en code Pyret.
4. Convertissez le second Cercle en code Pyret.

Notes pour l'enseignant·e

Insister sur la *lecture* du Cercle : on commence par la fonction la plus extérieure, qui consomme le résultat de la fonction intérieure. C'est exactement la structure d'une expression imbriquée en mathématiques : $f(g(x))$.

Interface du correcteur orthographique

Pyret connaît bien plus que des `Number`, `String`, `Boolean` et `Table` : il sait aussi produire des **Images**. Nous allons construire une petite interface visuelle pour notre correcteur.

Activité

1. Quels sont les deux *premiers* mots de votre table `resultats`?
2. Quel code permet d'extraire la colonne "word" de la *première ligne* de la table?

Voici quelques contrats (et exemples !) pour de nouvelles fonctions :

Contrat	Exemple
<code>string-length :: String -> Number</code>	<code>string-length("bonjour")</code> → 7
<code>text :: (String, Number, String) -> Image</code>	<code>text("IA !", 50,</code> <code>"blue")</code>

Activité

5. `string-length("bonjour")` renvoie? Que fait `string-length`? Elle compte le nombre de caractères d'une chaîne.
6. **Prédisez avant de taper** : pour chaque expression ci-dessous, que pensez-vous que Pyret va renvoyer? Tapez-les ensuite pour vérifier.

```
string-length(row-n(resultats, 1)["word"])
string-length(row-n(resultats, 2)["word"])
```

7. Évaluez `text("IA !", 50, "blue")`. Qu'obtenez-vous?
8. Quel code permet d'afficher le mot de la *deuxième ligne* de `resultats` en grandes lettres bleues?

La plupart des fonctions ci-dessous ont leur contrat *incomplet* : à vous d'expérimenter dans Pyret pour compléter les parties manquantes !

Nom	Domaine	Image
<code>triangle</code>	Number, String, String	Image
<code>star</code>		
<code>circle</code>		
<code>rectangle</code>		
<code>ellipse</code>		
<code>regular-polygon</code>		
<code>overlay</code>		
<code>rotate</code>		

Activité

5. Le Cercle d'Évaluation ci-dessous superpose le mot "Platon" au-dessus d'un rectangle gris plein. Convertissez-le en code et tapez-le dans Pyret :

```
(overlay
  (text "Platon" 20 "black")
  (rectangle 150 50 "solid" "gray"))
```

En notation Cercles imbriqués : `(overlay (text "Platon" 20 "black") (rectangle 150 50 "solid" "gray"))`.

Point clé

À vous de créer! Utilisez les contrats ci-dessus pour ajouter de la couleur, plusieurs formes (et peut-être même plusieurs suggestions!) à votre interface.

Notes pour l'enseignant·e

L'exploration des images est une activité à *seuil bas / plafond haut* qui devrait intéresser tout le monde. Certains élèves trouveront *tous* les contrats, la plupart ne le feront pas. Dès que le premier élève a réussi à utiliser la fonction `text`, donnez des indications sur les fonctions à prioriser avec le temps restant.

Ne pas essayer de garder tous les élèves au même rythme. Ce qui compte, c'est que *chacun·e* ait l'occasion d'explorer.

Synthèse

Réflexion

- Pourquoi peut-il être utile de séparer les étapes de *réflexion* et de *codage*? Pourquoi ne pas tout faire en même temps, tout le temps?
 - Les Cercles nous libèrent de la syntaxe pour se concentrer sur la *structure*. Une fois la structure claire, la traduction en code est mécanique.

Point clé

Composer des fonctions, c'est utiliser le *résultat* d'une fonction comme *argument* d'une autre. Les Cercles d'Évaluation sont la notation visuelle de cette composition : le Cercle intérieur produit la valeur que consomme le Cercle extérieur.

Annexe A : Table des contrats

Voici l'ensemble des contrats utilisés dans cette leçon :

Fonction	Contrat
sort	Table, String, Boolean -> Table
first-n-rows	Table, Number -> Table
last-n-rows	Table, Number -> Table
count	Table, String -> Table (table de fréquence)
bar-chart	Table, String -> Image
pie-chart	Table, String -> Image
row-n	Table, Number -> Row
string-length	String -> Number
text	String, Number, String -> Image
triangle	Number, String, String -> Image

Pour compléter les contrats des fonctions d'image (`star`, `circle`, `rectangle`, `ellipse`, `regular-polygon`, `overlay`, `rotate`), ouvrez la *documentation Pyret* ou consultez la leçon *Contracts for Strings and Images* de Bootstrap World (Fall 2026).

Annexe B : Exemple français « plation »

Dans la leçon originale, on utilise `alt-words("plation", WORDS-L, 4)` sur le dictionnaire anglais pour obtenir une riche table de suggestions. Dans notre adaptation française, "plation" est évalué contre WORDS-L-FR (40 000 mots français); la table `resultats` contiendra donc les suggestions les plus proches selon le dictionnaire français, par exemple :

- À distance 1 : `plation` lui-même s'il apparaît dans le dico, sinon très peu de mots (peut-être "platon", prénom du philosophe).
- À distance 2-3 : des mots comme "platine", "positions", "ration", etc.
- À distance 4 : une explosion combinatoire, d'où l'intérêt de *trier* puis *raccourcir* pour ne garder que les suggestions les plus pertinentes.

Si vous voulez expérimenter avec des mots français plus familiers, essayez :

```
resultats = alt-words("maizon", WORDS-L-FR, 3)
resultats = alt-words("ortograp", WORDS-L-FR, 4)
resultats = alt-words("ecole", WORDS-L-FR, 1)
```

Annexe C : Fichiers du projet

Fichier	Description
<code>Correcteur-Ortho-graphique-Starter.arr</code>	Fichier de démarrage Pyret (≈ 20 Ko) : bibliothèque Bootstrap + adaptations FR (alphabet ALPHABET-FR-SIMPLE, dictionnaires WORDS-XS-FR inlined et WORDS-L-FR importé).
<code>programmer-suggestions-ortho.tex</code>	Document de cours (ce fichier)
<code>images/screenshot-texto.png</code>	Capture d'une appli de messagerie proposant des alternatives
<code>images/CCbadge.png</code>	Logo Creative Commons

À noter : ce fichier de démarrage est le *même* que celui de la leçon précédente (*Algorithmes pilotés par les données*). Aucune modification n'est nécessaire : il expose déjà `alt-words(s, dict, n) -> Table`, les types `BKNode` pour les dictionnaires, et `WORDS-L-FR` importé automatiquement. Toutes les fonctions `table` de la bibliothèque Bootstrap (`sort`, `first-n-rows`, `count`, `bar-chart`, `pie-chart`, `row-n`, etc.) sont disponibles via le `use context url-file(...)` qui charge `core.arr`.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported. Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/programming-spelling-suggestions/>